

마이크로프로세서응용시스템 최종 프로젝트



Tx Rx Signal Limitation

```
#include <SoftwareSerial.h>
#include <AccelStepper.h>
#include <math.h>
```

```
#if F_CPU == 16000000UL
// hardcoded exception for compatibility with the bootloader shipped
// with the Duemilanove and previous boards and the firmware on the 8U2
// on the Uno and Mega 2560.
if (baud == 57600)
    use_u2x = false;
#endif

try_again:
if (use_u2x)
{
    _SFR_IO8(ucsrA) = 1 << u2x;
    baud_setting = (F_CPU / 4 / baud - 1) / 2;
}
else
{
    _SFR_IO8(ucsrA) = 0;
    baud_setting = (F_CPU / 8 / baud - 1) / 2;
}

if ((baud_setting > 4095) && use_u2x)
{
    use_u2x = false;
    goto try_again;
}

// assign the baud_setting, a.k.a. ubrr (USART Baud Rate Register)
_SFR_IO8(ubrrh) = baud_setting >> 8;
_SFR_IO8(ubrrl) = baud_setting;

_SFR_IO8(ucsrB) |= UCSRB_MASK;
_SFR_IO8(ucsrB) &= ~UDRE;
}
```

Atmega328P의 한계상 TfminiS와의 UART 통신이 Software를 통해 구현이 되어있어, 하드웨어적으로 구현되어 있는게 아님.

Why?

Atmega328P의 datasheet에 따르면, Hardware Serial은 USART0를 통해 구현할 수 있음.

해당 USART0는 HardwareSerial 라이브러리를 참고하면,

이미 해당 레지스터를 아두이노와 PC 통신에 이용중임을 알수 있으며, 해당 USART0는 한번에 기기 한대 밖에 이용 못하기에 SoftwareSerial를 이용해 TFminiS와 연결할 수 밖에 없음.

SoftwareSerial

```
// Wait approximately 1/2 of a bit
tunedDelay(_rx_delay_centering);
DebugPulse(_DEBUG_PIN2, 1);

// Read each of the 8 bits
for (uint8_t i=8; i > 0; --i)
{
    tunedDelay(_rx_delay_intrabit);
    d >>= 1;
    DebugPulse(_DEBUG_PIN2, 1);
    if (rx_pin_read())
        d |= 0x80;
}

// skip the stop bit
tunedDelay(_rx_delay_stopbit);
DebugPulse(_DEBUG_PIN1, 1);
```

```
_delay_loop_2(uint16_t __count)
{
    __asm__ volatile (
        "1: sbiw %0,1" "\n\t"
        "brne 1b"
        : "=w" (__count)
        : "0" (__count)
    );
}
```

SoftwareSerial에서 해당 for문을 이용해서 비트를 읽어오는데, `_rx_delay_intrabit*8`만큼의 `tunedDelay`가 발생함.

`tunedDelay`는 `_delay_loop_2`를 이용하는데, 해당 기능은, `delay_basic`에 어셈블리어를 통해 구현되어 있음. (해당 부분에 대해서만 ai에게 해석을 부탁함)

해당 코드에서는 Polling 상태로 `4__count` 클럭 만큼의 지연을 만듦.

따라서, 최종적으로 $4 \times (_rx_delay_centering + 8 \times _rx_delay_intrabit + _rx_delay_stopbit)$ 클럭사이클 만큼의 지연이 1바이트가 들어올 때마다 발생함.

SoftwareSerial

```
uint16_t SoftwareSerial::subtract_cap(uint16_t num, uint16_t sub) {  
    if (num > sub)  
        return num - sub;  
    else  
        return 1;  
}
```

```
// We already wait for 1 bit time = 23 cycles, so here we wait for  
// 0.5 bit time = (71 + 18 - 22) cycles.  
_rx_delay_centering = subtract_cap(bit_delay / 2, (4 + 4 + 75 + 17 - 23) / 4);  
  
// There are 23 cycles in each loop iteration (excluding the delay)  
_rx_delay_intrabit = subtract_cap(bit_delay, 23 / 4);  
  
// There are 37 cycles from the last bit read to the start of  
// stopbit delay and 11 cycles from the delay until the interrupt  
// mask is enabled again (which _must_ happen during the stopbit).  
// This delay aims at 3/4 of a bit time, meaning the end of the  
// delay will be at 1/4th of the stopbit. This allows some extra  
// time for ISR cleanup, which makes 115200 baud at 16Mhz work more  
// reliably  
_rx_delay_stopbit = subtract_cap(bit_delay * 3 / 4, (37 + 11) / 4);  
#else // Timings counted from gcc 4.3.2 output  
// Note that this code is a _lot_ slower, mostly due to bad register  
// allocation choices of gcc. This works up to 57600 on 16Mhz and  
// 38400 on 8Mhz.  
_rx_delay_centering = subtract_cap(bit_delay / 2, (4 + 4 + 97 + 29 - 11) / 4);  
_rx_delay_intrabit = subtract_cap(bit_delay, 23 / 4);  
_rx_delay_stopbit = subtract_cap(bit_delay, 23 / 4);
```

해당 계산식에 의거하여, TfminiS의 기본 baud rate인 115200와 19200에서의 값을 비교하면,

수치	115200	19200
bit_delay	34	308
_rx_delay_centering	1(문제점)	85
_rx_delay_intrabit	29	203
_rx_delay_stopbit	13	144
총합	246loop	1853loop

115200일때 centering delay가 -2이어야 하지만, delay가 unsigned int 이므로, 1로 적용시킴. 따라서, 구동이 가능한거지, 의도된 타이밍에 어긋나게 됨.

따라서, 7배정도의 클럭사이클 지연을 만들더라도, 정확한 타이밍에 맞추어 작동하도록 설계

Tfmini-S Datasheet

Table 12 General Parameter Configuration and Description

Parameters	Command	Response	Remark	Default setting
Obtain firmware version	5A 04 01 5F	5A 07 01 V1 V2 V3 SU	Version V3.2.1	
System reset	5A 04 02 60④	5A 05 02 00 61	Succeeded	
		5A 05 02 01 62	Failed	
Frame rate	5A 06 03 LL HH SU	5A 06 03 LL HH SU	1-1000Hz①	
Trigger detection	5A 04 04 62	Data frame	After setting the frame rate to 0, detection can be triggered with this command	
Output format	5A 05 05 01 65	5A 05 05 01 65	Standard 9 bytes(cm)	
	5A 05 05 02 66	5A 05 05 02 66	Pixhawk	
	5A 05 05 06 6A	5A 05 05 06 6A	Standard 9 bytes (mm)	
Baud rate	5A 08 06 H1 H2 H3 H4 SU	5A 08 06 H1 H2 H3 H4 SU	Set baud rate② E.g. 256000(DEC)=3E800(HEX),	115200

Tx Rx 통신을 통해 Baud rate를 115200 에서 19200으로 낮춤.
5A 08 06 00 4B 00 00 B3으로 적용해야 하므로, 해당 코드를 이용함.

```
#include <SoftwareSerial.h>

#define TFMINI_RX 9
#define TFMINI_TX 8

SoftwareSerial tf(TFMINI_RX, TFMINI_TX);

void setup() {
    Serial.begin(115200);
    tf.begin(115200UL); delay(100);
    uint8_t baud[8] = {0x5A, 0x08, 0x06, 0x00, 0x4B, 0x00, 0x00, 0xB3};
    tf.write(baud, 8);
    delay(200);
    uint8_t save[4] = {0x5A, 0x04, 0x11, 0x6F};
    tf.write(save, 4);
    delay(100);
}
```

스캐닝 기법

Single point LiDAR를 이용해서 3D 스캐닝을 진행하기 위해 턴테이블과 스텝모터를 통한 라이다의 각도 조절이 필요함.

그러나, 각도 조절을 해서 라이다를 조절하면 스텝모터의 지속적인 움직임을 보장하지 않고, 움직임을 일정치 않은 진동으로 인한 측정 오차가 발생함. 따라서, 스텝모터를 지속적으로 진동($0^{\circ}\sim 45^{\circ}$)시키며, 턴테이블을 일정한 속도로 움직이게 하며, 움직이는 주기를 서로 비틀어, 주기차이(20:23)를 이용해 스캔함.

또한, 턴테이블의 일정한 속도 유지가 중요하기에 PID 제어에서 Closed-loop 제어를 엔코더를 External Interrupt 신호로 처리하여 ISR을 통한 우선 변수 수정을 진행함.

이렇게 되면, 턴테이블의 PID 제어의 난이도가 높지 않음. (초기 Kick 이후, 등속 유지를 위해 작동하므로, 작동의 범위가 크지 않고, 계산을 encoder 기반으로 진행하기에, 속도 변화 오차가 국소적임)

Appendix

PID 오차 제어

▪ [구현] 아두이노

- 턴테이블 회전량을 PID 제어

```
// -----  
// PID loop for turntable (~200 Hz)  
// -----  
unsigned long now = millis();  
float dt = (now - prevTime) / 1000.0f;  
if (dt < 0.005f) return;  
prevTime = now;  
  
long currentCount = myEnc.read();  
long error = targetCount - currentCount;
```

↑ PID 제어 주기 & error 계산

해당 실습 수업때 진행한 코드는 PID 제어의 오차를 encoder count를 기반으로 하여 계산함.

이를 각속도 기반으로 수정하여 제작함.

```
void updateVelPid() {  
    unsigned long now = micros();  
    float dt = (now - pidPrevUs) * 1e-6f;  
    if (dt < 0.005f) return;  
    pidPrevUs = now;  
  
    long c = readEncCount();  
    float vel = (float)(c - pidPrevCount) / dt;  
    pidPrevCount = c;  
  
    float err = TARGET_CPS - vel;  
    velIntegral += err * dt;  
    if (velIntegral > INTEGRAL_MAX) velIntegral = INTEGRAL_MAX;  
    if (velIntegral < -INTEGRAL_MAX) velIntegral = -INTEGRAL_MAX;  
    float deriv = (err - prevErr) / dt;  
    prevErr = err;  
  
    float out = Kp * err + Ki * velIntegral + Kd * deriv;  
    if (out < 0) out = 0; // 정방향 전용  
    if (out > PWM_MAX) out = PWM_MAX;  
    driveMotor((int)out);  
}
```


턴테이블 모터 제어

```
//DC 모터 구동
void driveMotor(int pwm) {
    pwm *= MOTOR_DIRECTION;
    bool forward = (pwm >= 0);
    int mag = pwm;
    if (forward < 0) mag *= -1;
    if (mag > PWM_MAX) mag = PWM_MAX;
    if (mag > 0 && mag < PWM_MIN_KICK) mag = PWM_MIN_KICK;
    if (forward) { digitalWrite(DC_IN3, LOW); digitalWrite(DC_IN4, HIGH); }
    else        { digitalWrite(DC_IN3, HIGH); digitalWrite(DC_IN4, LOW); }
    analogWrite(DC_ENB, mag);
}
```

Pwm 제어값의 min, max, 방향 보정후 적용.

스텝모터 회전

```
void updateTilt() {
    static unsigned long lastUs = 0;
    unsigned long now = micros();
    if (now - lastUs >= 3000) {
        lastUs = now;
        float panRevs = (float)readEncCount() / COUNTS_PER_TABLE_REV;
        float phase = panRevs * TILT_CYCLES_PER_REV;
        phase -= floorf(phase);
        if (phase < 0.0f) phase += 1.0f;
        float frac;
        if (phase < 0.5f) {
            frac = phase * 2.0f;
        } else {
            frac = (1.0f - phase) * 2.0f;
        }
        long target = (long)(TILT_MAX_STEPS * frac) * TILT_DIRECTION;
        tiltStepper.moveTo(target);
    }
    tiltStepper.run();
}
```

- 3ms 마다 step을 재계산
- panRevs에서 엔코더 수 기반 몇바퀴 회전인지 계산
- Phase에서 회전수 대비 tilt 위치 계산
- Phase는 몇번째 회전인지 안중요하므로, 소수부분만 남김
- 이후, 상승 하강 여부를 0.5 기준 비교후 step 모터에 목표 설정
- If 문 밖의 run을 통해 step 모터의 부드러운 작동을 보장+계산량 감소

엔코더 계산

```
//엔코더
static const int8_t PRECALC[16] = { 0, -1, 1, 0, 1, 0, 0, -1, -1, 0, 0, 1, 0, 1, -1, 0 };
volatile long encoder_count = 0;
volatile uint8_t encState = 0;
▼ static inline void encoder_interrupts() {
    uint8_t p = PIND;
    uint8_t s = (((p >> ENC_A) & 1) << 1) | ((p >> ENC_B) & 1); // (A<<1)|B
    encoder_count += PRECALC[(encState << 2) | s];
    encState = s;
}
ISR(INT0_vect) { encoder_interrupts(); } // D2(A)
ISR(INT1_vect) { encoder_interrupts(); } // D3(B)

▼ long read_encoder_count() {
    noInterrupts();
    long c = encoder_count;
    interrupts();
    return c * ENCODER_DIRECTION;
}
```

- 엔코더 신호를 인터럽트로 활용
- 엔코더 핀이 두개이므로, 둘다 ISR로 처리
- ENC_A,ENC_B를 이전 값과 비교하여 (00,01,10,11) 총 16가지의 경우에 해당하는 값을 미리 계산해 ISR 내의 지연을 최소화하기 위함
- 그러나, read_encoder_count()에서는 읽는 순간 ISR이 발동하여 값이 갱신되면 안되므로, noInterrupts 처리를 통해 갱신X

Python GUI

```
def connect(p : {__getitem__, Serial} ): 1 usage
    sp, auto = p["serial"], p["autostop"]
    try:
        ser = serial.Serial(sp["port"], int(sp["baud"]), timeout=0.1)
    except Exception as e:
        sys.exit(f"[에러] 시리얼 연결 실패: {e}")
    print(f"[연결] {sp['port']} @ {sp['baud']} - 부팅 대기(2s)...")
    time.sleep(2.0)
    try:
        ser.reset_input_buffer()
    except Exception:
        pass

    ser.write(b"z\n")
    time.sleep(0.2)
    ser.write(b"s\n")
    print("시작 명령 전송 - 스캔 중")

    raw_list = []
    last_print, last_enc = 0.0, 0.0
    last_line = "(대기)"
```

- Connect 함수를 통해 Arduino에서 Serial 통신을 진행함
- 실습 예제에서 참고하여 작성했으나, params.yaml을 이용하기 위해 변수정도만 일부 수정
- 이후 내용 또한 실습 예제 기반으로 작성

XYZ 좌표계 변환-1

도구 내용

```
# Data line: angle_deg,distance_cm,strength
parts = line.split(",")
if len(parts) != 3:
    continue
try:
    ang = float(parts[0])
    dist = float(parts[1])
    strength = float(parts[2])
```

```
line = ser.readline().decode("utf-8", errors="ignore").strip()
if line:
    if line.startswith("#"):
        sys.stdout.write("\r" + " " * 88 + "\r")
        print(f"[EVENT] {line.lstrip('# ').strip()}")
    else:
        parts = line.split(",")
        if len(parts) == 4:
            try:
                rec = tuple(float(x) for x in parts)
                raw_list.append(rec)
                last_enc = rec[1]
                last_line = (f"tilt {rec[0]:5.1f}° enc {rec[1]:7.0f} "
                             f"d {rec[2]:4.0f}cm s {rec[3]:4.0f}")
            except ValueError:
                pass
```

- 실습 예제에서는 ang dist strength 정보가 필요
- Z 좌표계를 위해 현재 tilt 각도를 추가함.

XYZ 좌표계 변환-2

- 도구 내용
- 극 좌표계에서 point 위치 $(\theta_1, r)_t$ 를 $(x_w, y_w)_t$ 로 변환 함수
- 현재는 2D 변환만 구현

```
def polar_to_object_xy(
    angle_deg: np.ndarray,
    dist_cm: np.ndarray,
    d_axis_cm: float,
) -> tuple[np.ndarray, np.ndarray]:
    """Convert turntable (angle, distance) pairs to object-frame Cartesian.

    Args:
        angle_deg: Turntable rotation angles [deg].
        dist_cm: TFmini distance readings [cm].
        d_axis_cm: Sensor-to-rotation-axis distance [cm].

    Returns:
        Tuple (x, y) in cm, centered on the rotation axis (object frame).
    """
    r = d_axis_cm - dist_cm          # radius from axis to surface point
    theta = np.deg2rad(angle_deg)
    x = r * np.cos(theta)
    y = r * np.sin(theta)
```

```
mask = (dist >= flt["dist_min"]) & (dist <= flt["dist_max"]) \
        & (strg >= flt["strength_min"])
sub = raw[mask]
phi = np.radians(sub[:, 0])
theta = np.radians(sub[:, 1] * 360.0 / CPR) # enc 카운트 -> 턴테이블 각도로 변환
radial = g["D"] - sub[:, 2] * np.cos(phi) # radial: x/y 계산에 필요(필터 아님)
xyz = np.column_stack([radial * np.cos(theta),
                       -radial * np.sin(theta),
                       g["H"] + sub[:, 2] * np.sin(phi)]) #실습 예제에서 phi를 이용해 Z축 추가
```

- 실습 예제에서는 2차원 변환만 진행됨
- Z 축을 위해 현재 tilt 각도를 이용하여 Z값을 계산

팀원 파트 분배

- 박경민 – 총괄, 코드 정리, 파라미터화, 인터럽트 수정
- 정원호 – 아두이노 코드 초안 작성, 하드웨어 제작
- 전종민 – 파이썬 코드 초안 작성, 하드웨어 제작

출처

- **(Serial Library)** <https://github.com/arduino/ArduinoCore-avr/blob/master/cores/arduino/HardwareSerial.cpp>
- **(SoftwareSerial Library)** <https://github.com/arduino/ArduinoCore-avr/blob/master/libraries/SoftwareSerial/src/SoftwareSerial.cpp>
- **delay_basic** (Arduino15/packages/arduino/tools/avr-gcc/7.3.0-atmel3.6.1-arduino7/avr/include/util)
- **(TFminiS datasheet)** <https://en.benewake.com/uploadfiles/2024/04/20240426140053930.pdf>

- **전체 코드** (https://github.com/PARKasd/2026_Microprocessor_project)